# `Intermediate.lean` 逐步说明

从因子求和到 Dirichlet 卷积与 Möbius 反演

本文对同目录的 `Intermediate.lean` 作逐步说明。读者可先看通常的数学证明，再将它与 Lean 4 的每条指令对照，最后在 VS Code 中观察证明状态。本例不要求读者先学习一门系统的 Lean 课程，但需熟悉素因子分解、有限求和及对数乘法公式。若已读过 3–4–1 例程，可把这里的重点放在归纳假设、函数相等与卷积反演上。

## 1 要证明什么，采用哪些基础结果

对正整数 $n$，本例证明

$$\sum_{d\mid n}\Lambda(d)=\log n, \tag{1}$$

并由此推出

$$\Lambda(n)=\sum_{d\mid n}\mu(d)\log(n/d). \tag{2}$$

这里 $\Lambda(p^k)=\log p$（$p$ 为素数、$k\ge 1$），其他正整数处取零。本例中的 $\mu$ 是 Möbius 函数，log 为实自然对数。

证明分成四个命题：因子和恒等式、卷积等式、卷积反演、反演的有限和形式。代码不直接调用库中已经完成的 `vonMangoldt_sum`，也不直接调用 $\Lambda$ 的现成 Möbius 反演公式，而是写出上述推理过程。

采用的基础输入是：自然数的素因子分解归纳，素数幂的因子枚举，互素数的素数幂因子分离，实对数的乘法与乘方公式，以及一般 Dirichlet 卷积理论和 Möbius 逆关系。这些基础结果来自 Mathlib，本例不重新证明它们。导入一个包含目标定理的模块，不等于实际调用了那个目标定理；应查看证明中使用的具体名称。

### 1.1 先用通常数学语言证明因子和

记 $S(n)=\sum_{d\mid n}\Lambda(d)$。当 $n=p^k$ 时，正因子是 $1,p,\dots,p^k$，于是

$$S(p^k)=\Lambda(1)+\sum_{j=1}^{k}\Lambda(p^j)=k\log p=\log(p^k).$$

这也覆盖 $k=0$、即 $n=1$ 的情形，此时求和为零。

若 $(a,b)=1$，则 $ab$ 的每个素数幂因子完全来自 $a$ 或完全来自 $b$。理由是：素数不可能同时整除互素的 $a,b$；由素因子分解，$ab$ 中该素数的全部指数只能来自其中一个因子。两组素数幂因子不相交。因此

$$S(ab)=S(a)+S(b).$$

假定已知 $S(a)=\log a$、$S(b)=\log b$，即得

$$S(ab)=\log a+\log b=\log(ab).$$

每个正整数都能分解为两两互素的素数幂，故归纳得到式 (1)。代码使用一般归纳原理组织这一论证，而不自行重写素因子分解理论。

### 1.2 再用卷积完成反演

对算术函数 $f,g$，Dirichlet 卷积定义为

$$(f*g)(n)=\sum_{de=n} f(d)g(e).$$

以下用 $\mathbf{u}(n)=1$（$n\geq 1$）表示常值算术函数，用 $\varepsilon$ 表示卷积单位元：

$$\varepsilon(1)=1,\qquad \varepsilon(n)=0\quad (n>1).$$

写 $\ell(n)=\log n$，则因子和等式就是 $\Lambda*\mathbf{u}=\ell$。利用一般逆关系 $\mathbf{u}*\mu=\varepsilon$，逐步计算

$$\begin{aligned}\mu*\ell=\ell*\mu&=(\Lambda*\mathbf{u})*\mu\\&=\Lambda*(\mathbf{u}*\mu)=\Lambda*\varepsilon=\Lambda.\end{aligned}$$

展开左边，即得式 (2)。这里的交换律、结合律与单位元都是卷积的性质，不是把算术函数按点相乘后的普通代数规则。

## 2 完整源码

下面直接读取同目录的实际程序。行号用于对照，不属于程序内容。

```
/-
The divisor-sum proof below is adapted from ArithmeticFunction.vonMangoldt_sum
in Mathlib/NumberTheory/VonMangoldt.lean, copyright (c) 2022 Bhavik Mehta.
This file is distributed under Apache-2.0; see Apache-2.0.txt in this directory.
The adaptation expands the proof into an annotated learning example.
-/
import Mathlib.NumberTheory.VonMangoldt
import Mathlib.Tactic

/-!
# 中等示例：von Mangoldt 恒等式与 Möbius 反演

实际证明分为三段：按素数幂与互素乘积归纳证明因子和恒等式；
将它写成卷积；再用 Möbius 函数与常数算术函数的卷积逆关系反演。
不调用 vonMangoldt_sum 或已经算好 Λ 的 Möbius 公式。

基础输入：整数的素因子分解归纳、素数幂的因子枚举、互素数的
素数幂因子分离、对数乘法公式、一般 Dirichlet 卷积及其 Möbius 逆。
这些一般结果不是本例声称重新证明的内容。
-/

open ArithmeticFunction Finset
open scoped ArithmeticFunction
```

```
24
25  namespace Progressive
26
27  -- 第一段：证明所有正整数的因子和恒等式。
28  theorem mangoldt_divisor_sum (n : ℕ) :
29      ∑ d ∈ n.divisors, Λ d = Real.log n := by
30    -- 一般素因子分解归纳：处理 0、1，素数幂，以及互素乘积。
31    -- 库的 log(0) 和 divisors(0) 约定使 n=0 也成立；数学用途是 n≥1。
32    refine Nat.recOnPrimeCoprime ?_ ?_ ?_ n
33    -- 基础情形由 Λ(1)=0、空因子集及 log(1)=0 等定义化简得到。
34    · simp
35    -- 素数幂情形：p^k 的因子为 1,p,...,p^k。
36    · intro p k hp
37      -- 枚举因子，把 log(p^k) 改写成 k log p，并单独取出 Λ(1)。
38      rw [Nat.sum_divisors_prime_pow hp, Nat.cast_pow, Real.log_pow,
39        Finset.sum_range_succ', Nat.pow_zero, vonMangoldt_apply_one]
40      -- 每个正次幂 p^(j+1) 的 Λ 值都是 log p；有限常数和为 k log p。
41      simp [vonMangoldt_apply_pow (Nat.succ_ne_zero _), vonMangoldt_apply_prime hp]
42    -- 互素乘积情形：已知 a、b 的结论，证明 ab 的结论。
43    · intro a b haPositive hbPositive hab ha hb
44      -- 按 Λ 的定义，只保留因子中的素数幂；其他因子贡献为零。
45      simp only [vonMangoldt_apply, ← Finset.sum_filter] at ha hb ⊢
46      -- a、b 互素，所以 ab 的素数幂因子来自 a 或 b，且两类不交。
47      rw [Nat.mul_divisors_filter_prime_pow hab, Finset.filter_union,
48        Finset.sum_union (Nat.disjoint_divisors_filter_isPrimePow hab)]
49      -- 对两个互不相交的因子集合分别使用归纳假设。
50      rw [ha, hb, Nat.cast_mul]
51      -- a、b 均为正，故可用 log(ab)=log a+log b；最后取等式的反向。
52      exact (Real.log_mul
53        (Nat.cast_ne_zero.mpr (pos_of_gt haPositive).ne')
54        (Nat.cast_ne_zero.mpr (pos_of_gt hbPositive).ne')).symm
55
56  -- 第二段：把已经证明的因子和改写成 Dirichlet 卷积。
57  -- 这里的 ζ 是恒等于 1（在正整数上）的算术函数，不是复变 ζ 函数。
58  theorem mangoldt_convolution : Λ * (ζ : ArithmeticFunction ℝ) = ArithmeticFunction.log := by
59    -- 两个算术函数相等，只需在每个自然数 n 处取值相等。
60    ext n
61    -- 与 ζ 卷积就是因子求和；展开后正是第一段的命题。
62    rw [ArithmeticFunction.coe_mul_zeta_apply]
63    exact mangoldt_divisor_sum n
64
65  -- 第三段：在卷积等式右边乘 μ，实际完成反演，而不引用 Λ 的反演结果。
66  theorem mangoldt_mobius_convolution :
67      (μ : ArithmeticFunction ℝ) * ArithmeticFunction.log = Λ := by
68    calc
69      -- 先交换两个因子的位置，以便随后使用结合律。
70      (μ : ArithmeticFunction ℝ) * ArithmeticFunction.log =
71          ArithmeticFunction.log * μ := mul_comm _ _
72      -- 将第二段得到的 log=Λ*ζ 代入。
73      _ = (Λ * ζ) * μ := by rw [mangoldt_convolution]
74      -- 卷积结合律允许先计算 ζ*μ。
75      _ = Λ * (ζ * μ) := mul_assoc _ _ _
76      -- 使用一般 Möbius 逆关系 ζ*μ=1；不是调用本例的目标定理。
77      _ = Λ * 1 := by rw [ArithmeticFunction.coe_zeta_mul_coe_moebius]
78      -- 卷积单位元消去，得到 Λ。
79      _ = Λ := mul_one _
```

```
80
81  -- 最后把函数等式展开为有限和，便于和通常的数学写法对照。
82  theorem mangoldt_mobius_sum (n : ℕ) :
83      ∑ p ∈ n.divisorsAntidiagonal, (μ p.1 : ℝ) * Real.log p.2 = Λ n := by
84    -- 对第三段的算术函数等式，在同一个 n 处取值。
85    have h := congrArg (fun f : ArithmeticFunction ℝ => f n) mangoldt_mobius_convolution
86    -- 展开卷积定义；有序因子对 (d,e) 满足 de=n，故这就是 Σ μ(d)log(n/d)。
87    simpa only [ArithmeticFunction.mul_apply, ArithmeticFunction.intCoe_apply,
88      ArithmeticFunction.log_apply] using h
89
90  end Progressive
```

# 3 文件准备与基本记号

## 3.1 导入与来源说明

```
import Mathlib.NumberTheory.VonMangoldt
import Mathlib.Tactic
```

第一行导入 von Mangoldt 函数及有关算术函数理论；第二行提供证明策略。文件顶部的英文注释说明：因子和证明改编自 Bhavik Mehta 编写的 Mathlib 源文件，按 Apache-2.0 发布。本例的作用是把已有证明展开为附中文注释的学习材料，不将它当作新的数学证明方法。许可全文与来源说明在同目录中保留。

## 3.2 打开名称与记号作用域

```
open ArithmeticFunction Finset
open scoped ArithmeticFunction
namespace Progressive
```

`open` 使有关名字空间中的名称可以简写；`open scoped` 启用相应记号，使程序能使用 $\Lambda, \mu, \zeta$ 等符号。`namespace Progressive` 为后续定理添加名称前缀。例如第一个定理的完整名称是 `Progressive.mangoldt_divisor_sum`。这个前缀与当前目录名不同，并不影响构建。最后的 `end Progressive` 结束名字空间。

## 3.3 自然数、实数与类型转换

`n : ℕ` 表示 $n$ 是自然数；`Real.log` 的输入却是实数。当代码写 `Real.log n` 时，Lean 会插入从自然数到实数的类型转换。`Nat.cast_pow`、`Nat.cast_mul` 分别把转换后的乘方、乘积改写为实数乘方、实数乘积。这些步骤使自然数上的因子关系与实数上的对数公式能够衔接，不是新增的数学假设。

## 3.4 两个不能混淆的符号

程序中的 `ζ : ArithmeticFunction ℝ` 是正整数处恒为 1、零点处取 0 的算术函数，对应上文的 $\mathbf{u}$，不是解析数论中的复变 $\zeta(s)$。`ArithmeticFunction ℝ` 表示取实数值、在 0 处为零的算术函数类型。

在这个类型中，* 表示 Dirichlet 卷积，1 表示卷积单位元 $\varepsilon$。例如 $\zeta(2) = 1$，而卷积单位元在 2 处取 0。将两者混为一谈，会误读反演的最后两步。

# 4 第一段：因子和恒等式

## 4.1 定理陈述与有限求和

```
theorem mangoldt_divisor_sum (n : ℕ) :
    ∑ d ∈ n.divisors, Λ d = Real.log n := by
```

theorem 声明定理；冒号后是目标；:= by 开始策略式证明。n.divisors 是正因子的有限集合。∑ d ∈ n.divisors, Λ d 是在该集合上求和，不是无穷级数，也不涉及收敛问题。对于 $n \geq 1$，目标正是式 (1)。

Mathlib 约定 Nat.divisors 0 为空集、Real.log 0 = 0，且算术函数在 0 处取零。因此程序定理还覆盖 $n = 0$，但这只是库的全定义约定，不能据此把通常实对数理解为在零点有定义。本书中的数论应用仍取 $n \geq 1$。

## 4.2 用素因子分解归纳拆出三个目标

```
refine Nat.recOnPrimeCoprime ?_ ?_ ?_ n
```

refine 用一个尚有待填空的证明结构细化当前目标；三个 ?_ 是当前待证明的子目标，并不是允许保留到最后的证明缺口。缩进后的三个圆点分支逐一填满它们。若记 $P(n)$ 为上述因子和等式，三个任务依次为：

1. 证明 $P(0)$。
2. 对素数 $p$ 和自然数 $k$，证明 $P(p^k)$。
3. 对 $a > 1, b > 1, (a, b) = 1$，从 $P(a), P(b)$ 推出 $P(ab)$。

这里没有独立的 $P(1)$ 分支，因为 $1 = p^0$ 已包括在第二项中。源码注释概括提到 $0, 1$ 等基础情形；准确的三个接口应以上面的归纳原理为准。

## 4.3 零的分支：化简结束

```
· simp
```

圆点开始第一个子证明；simp 根据库中的化简规则，把空集求和及零点对数约定化为零，从而关闭该目标。这一行处理的是 $n = 0$；$n = 1$ 随素数幂分支处理。

## 4.4 素数幂分支：引入参数

```
· intro p k hp
```

intro 把待证的全称命题及前提引入当前上下文。p,k 是自然数；hp 是 $p$ 为素数的证明，不是第三个数。此时目标是 $\sum_{d \mid p^k} \Lambda(d) = \log(p^k)$。

### 4.5 逐项改写素数幂的因子和

```
rw [Nat.sum_divisors_prime_pow hp, Nat.cast_pow, Real.log_pow,
  Finset.sum_range_succ', Nat.pow_zero, vonMangoldt_apply_one]
```

`rw` 依次按方括号中的等式改写当前目标，各项作用如下。

**`Nat.sum_divisors_prime_pow hp`**
用 $p$ 为素数的条件，把因子和改写为 $\sum_{j=0}^{k}\Lambda(p^j)$。

**`Nat.cast_pow` 与 `Real.log_pow`**
将自然数乘方转到实数，并把 $\log(p^k)$ 改写为 $k\log p$。

**`Finset.sum_range_succ'`**
从长度为 $k+1$ 的有限和中取出索引为零的一项，其余写作 $j+1$，$0\le j<k$。

**`Nat.pow_zero` 与 `vonMangoldt_apply_one`**
用 $p^0=1$ 和 $\Lambda(1)=0$ 消去被取出的项。

改写后需要说明：余下 $k$ 个正次幂项之和等于 $k\log p$。

### 4.6 把每个正次幂项化为同一常数

```
simp [vonMangoldt_apply_pow (Nat.succ_ne_zero _),
  vonMangoldt_apply_prime hp]
```

`Nat.succ_ne_zero _` 证明后继自然数 $j+1$ 不为零，下划线表示参数由 Lean 推断。

`vonMangoldt_apply_pow` 用该条件把 $\Lambda(p^{j+1})$ 化为 $\Lambda(p)$；`vonMangoldt_apply_prime hp` 再把它化为 $\log p$。`simp` 同时计算有限常数和，完成本分支。当 $k=0$ 时，剩下的是空和，等式仍成立。

### 4.7 互素乘积分支：分清条件与归纳假设

```
· intro a b haPositive hbPositive hab ha hb
```

当前上下文的数学内容是

$$a>1,\quad b>1,\quad (a,b)=1,\quad S(a)=\log a,\quad S(b)=\log b.$$

`haPositive` 与 `hbPositive` 实际给出的是 $1<a$、$1<b$，比名字暗示的正性更强；`hab` 是互素性证明；`ha,hb` 是两个归纳假设。目标为 $S(ab)=\log(ab)$。这些局部名称可以自行选取，真正决定含义的是各自的类型。

### 4.8 只保留素数幂因子

```
simp only [vonMangoldt_apply, ← Finset.sum_filter] at ha hb ⊢
```

`vonMangoldt_apply` 展开 $\Lambda$：不是素数幂的输入处贡献零。反向使用 `Finset.sum_filter` 把“条件不满足时取零”的求和改写为对筛选后的有限集合求和。`←` 指明逆向改写；`only` 限定只使用列出的化简规则。`at ha hb ⊢` 表示同时改写两个归纳假设及当前目标，`⊢` 指目标。这不是把假设删去，而是把三处表达统一到能使用因子集合引理的形式。

### 4.9 分离两个互不相交的因子集合

```
rw [Nat.mul_divisors_filter_prime_pow hab, Finset.filter_union,
  Finset.sum_union (Nat.disjoint_divisors_filter_isPrimePow hab)]
```

第一项利用互素性，把 $ab$ 的素数幂因子集合拆为 $a, b$ 的相应集合之并。第二项把筛选运算分配到并集上，使表达式与归纳假设匹配。第三项利用这两个集合不交，将并集上的和拆为两个和。不交性是必要的：若集合有交，直接相加会重复计入交集。本例因子集中的 1 已被“素数幂”条件排除，所以互素数共有的因子 1 不构成障碍。

### 4.10 代入归纳假设并处理对数

```
rw [ha, hb, Nat.cast_mul]
exact (Real.log_mul
  (Nat.cast_ne_zero.mpr (pos_of_gt haPositive).ne')
  (Nat.cast_ne_zero.mpr (pos_of_gt hbPositive).ne')).symm
```

第一行将两个和替换成 $\log a, \log b$，并把自然数乘积的类型转换移入乘积。目标变为 $\log a + \log b = \log(ab)$。

`pos_of_gt haPositive` 从 $1 < a$ 得到 $0 < a$；`.ne'` 得到 $a \neq 0$；`Nat.cast_ne_zero.mpr` 再把自然数非零转为相应实数非零。$b$ 的处理相同。这些证明用于满足 `Real.log_mul` 的两个非零条件。该对数公式给出 $\log(ab) = \log a + \log b$，而目标方向相反，故使用 `.symm` 取对称等式。`exact` 把类型恰好等于当前目标的证明交给 Lean，完成最后一个归纳分支。

## 5 第二段：把因子和写成卷积

```
theorem mangoldt_convolution :
    Λ * (ζ : ArithmeticFunction ℝ) = ArithmeticFunction.log := by
  ext n
  rw [ArithmeticFunction.coe_mul_zeta_apply]
  exact mangoldt_divisor_sum n
```

这是算术函数的相等，不是某个固定 $n$ 处的相等。`ArithmeticFunction.log` 在 $n$ 处取值为 `Real.log n`。括号中的类型标注明确指定 $\zeta$ 的值域是实数。

`ext n` 使用外延性：只要在每个输入 $n$ 处取值相等，两个算术函数就相等。

`coe_mul_zeta_apply` 展开与 $\zeta$ 的卷积，得到因子和。此时目标正是第一段已证明的命题，最后一行直接应用它。这里引用的是本文件刚建立的结论，不是跳过第一段调用库中现成的目标恒等式。

## 6 第三段：实际完成 Möbius 反演

### 6.1 读出函数等式

```
theorem mangoldt_mobius_convolution :
    (μ : ArithmeticFunction ℝ) * ArithmeticFunction.log = Λ := by
```

Möbius 函数原本为整数值；类型标注把它视为实值算术函数，从而与实对数和实值 $\Lambda$ 卷积。目标是 $\mu * \ell = \Lambda$。

### 6.2 用 calc 写出等式链

```
calc
  (μ : ArithmeticFunction ℝ) * ArithmeticFunction.log =
      ArithmeticFunction.log * μ := mul_comm _ _
  _ = (Λ * ζ) * μ := by rw [mangoldt_convolution]
  _ = Λ * (ζ * μ) := mul_assoc _ _ _
  _ = Λ * 1 := by rw [ArithmeticFunction.coe_zeta_mul_coe_moebius]
  _ = Λ := mul_one _
```

`calc` 把长证明写成首尾相接的等式链。每行开头的下划线承接上一行的右端；每行 `:=` 后给出该步的依据。

1. `mul_comm` 使用实值算术函数卷积的交换律，先把 $\mu$ 移到右边。
2. 用第二段的 $\Lambda * \zeta = \ell$ 将 $\ell$ 换回卷积形式。虽然 `rw` 写的是正向规则，它在这个等式目标中将右侧的 $\Lambda * \zeta$ 化为 $\ell$，从而关闭该行的相等目标；数学阅读时可理解为把 $\ell$ 替换为 $\Lambda * \zeta$。
3. `mul_assoc` 使用结合律，使 $\zeta$ 与 $\mu$ 先卷积。
4. 一般 Möbius 逆关系将 $\zeta * \mu$ 替换为卷积单位元 1。
5. `mul_one` 用单位元律得到 $\Lambda$。

使用一般逆关系并不等于直接调用 $\Lambda$ 的反演结果：还须先识别因子和，建立第二段的卷积等式，再完成这条等式链。每行均有独立的已核验依据。

## 7 第四段：把函数等式还原为有限和

### 7.1 用有序因子对陈述结论

```
theorem mangoldt_mobius_sum (n : ℕ) :
    ∑ p ∈ n.divisorsAntidiagonal,
      (μ p.1 : ℝ) * Real.log p.2 = Λ n := by
```

`n.divisorsAntidiagonal` 枚举乘积为 $n$ 的有序因子对。这里 `p` 是一对自然数，`p.1`、`p.2` 是第一、第二分量，不是素数变量。对 $n \geq 1$，写成 $(d, e)$ 后满足 $de = n$，因而 $e = n/d$；每个正因子 $d$ 恰好对应一对。所以目标就是 $\sum_{d|n} \mu(d) \log(n/d) = \Lambda(n)$，与式 (2) 只差等号方向。

### 7.2 对函数等式取同一点的值

```
have h := congrArg (fun f : ArithmeticFunction ℝ => f n)
  mangoldt_mobius_convolution
```

`have h :=` 引入局部已证事实 $h$。`fun f : ArithmeticFunction ℝ => f n` 是“在固定 $n$ 处取值”的函数。`congrArg` 表示：若 $A = B$，则对两边应用同一函数后仍相等。因此 $h$ 的数学内容是 $(\mu * \ell)(n) = \Lambda(n)$。这一步把函数层面的等式送回数值层面，没有增加新假设。

### 7.3 展开定义，得到所需形式

```
simpa only [ArithmeticFunction.mul_apply, ArithmeticFunction.intCoe_apply,
  ArithmeticFunction.log_apply] using h
```

`mul_apply` 把卷积在 $n$ 处的值展开为有序因子对求和。`intCoe_apply` 说明整数值算术函数转为实值后，在某点取值就是把原整数值转为实数。`log_apply` 展开算术函数对数的取值。`simpa only [...] using h` 表示只用列出的规则化简 $h$ 及目标，然后把化简所得的证明用于目标。它并未再次证明一般 Möbius 反演，而是将上一段的具体反演等式展开。

### 7.4 用 n=12 对照各项的含义

第一段给出

$$\Lambda(1) + \Lambda(2) + \Lambda(3) + \Lambda(4) + \Lambda(6) + \Lambda(12) = 2\log 2 + \log 3 = \log 12.$$

第四段给出

$$\Lambda(12) = \log 12 - \log 6 - \log 4 + \log 2 = 0.$$

后一个和中 $d = 4, 12$ 的 Möbius 值为零，$d = 6$ 的项为 $+\log 2$。这两条等式说的是不同对象：前者对所有因子处的 $\Lambda$ 值求和，后者恢复单个输入 12 处的值。这个例子帮助核对理解，不能代替程序中对所有 $n$ 的证明。

## 8 在 VS Code 中运行与逐步观察

### 8.1 打开完整项目，而不是单独打开本目录

在 VS Code 中选择“文件—打开文件夹”，打开包含 `lakefile.toml`、`lean-toolchain` 的项目根目录：

```
/Users/lichengtong/lichengtong-RH/
  形式化证明-使用 Lean4 核验本书数学推导/
```

上面为便于排版分成两行，实际是同一完整路径。读者在自己的电脑上应使用自己的克隆位置。安装 Lean 4 扩展后，在资源管理器中进入“Lean 入门示例”目录并打开 `Intermediate.lean`。项目配置负责选定 Lean 和 Mathlib 版本。

按 `Cmd+Shift+P`（Windows/Linux 通常为 `Ctrl+Shift+P`）打开命令面板，选择名称中含 `Lean 4`、`InfoView`、`Toggle InfoView` 的命令。若当前扩展的菜单文字略有不同，可搜索 `InfoView`。

### 8.2 建议观察的五个位置

保存文件并等待检查完成后，将光标依次放到以下指令之后，查看右侧状态。

1. `refine Nat.recOnPrimeCoprime ...`：观察三个子目标怎样生成。
2. `intro a b ... ha hb`：观察正性、互素性及两个归纳假设的类型。
3. `rw [ha, hb, Nat.cast_mul]`：观察求和目标如何缩减为对数乘法公式。
4. 第二段的 `ext n`：观察函数相等如何变成逐点相等。
5. 第四段的 `have h := ...`：观察局部事实 $h$ 如何从第三段的函数等式产生。

具体显示会随光标位置、扩展版本和隐含参数展开程度变化。本文描述的是证明状态的数学含义，并非伪造的逐屏输出；读者应以自己编辑器中实际显示的目标为准。

### 8.3 如何判断检查通过

`Tactic state` 显示 `No goals`，表示光标所在证明位置已没有待解决的目标。它本身不证明整个文件都无错误。`Goals accomplished!` 若出现，同样应结合所属定理及位置理解。`Expected type` 显示的是当前位置表达式应具有的类型，有时会保留一个写成 `⊢ ...` 的命题；这不必然表示还有未完成目标，应与 `Tactic state` 区分。

确认整个文件时，还应等待检查结束，查看 Problems 或 Messages 中是否有错误，并用下节的命令核验。`No info found` 通常仅表示光标处没有可显示的证明状态，不能当作证明通过或失败的结论。若 Mathlib 未找到，应先核对是否打开了完整项目。

## 9 终端检查与证明依赖

### 9.1 单独检查本例

在项目根目录运行：

```
lake env lean Lean 入门示例/Intermediate.lean
```

`lake env` 设置项目环境，`lean` 检查指定文件。命令成功结束时，本例通常没有额外打印文本；应确认命令已结束且没有错误，不能仅凭暂时没有输出判断成功。若只出现依赖仓库 `has local changes` 的警告而命令正常退出，该警告不是证明错误，但说明本地依赖目录有改动，不能据此声称依赖与远端原版完全相同。

检查整个项目默认入口及其导入模块可运行：

```
lake build
```

该命令不自动检查目录中所有未被导入的私人练习文件。若出现找不到 Mathlib 的错误，请在项目根目录核对依赖安装；不要把更改 Lean 版本当作首选修复方法。

### 9.2 查看四个定理的公理依赖

先在项目根目录执行 `lake build`，再创建临时检查文件，写入：

```
import «Lean 入门示例 ».Intermediate

#print axioms Progressive.mangoldt_divisor_sum
#print axioms Progressive.mangoldt_convolution
#print axioms Progressive.mangoldt_mobius_convolution
#print axioms Progressive.mangoldt_mobius_sum
```

将临时文件放在该项目中，用 `lake env lean` 加上实际文件名运行。`##print axioms` 报告证明依赖的公理，不是请求读者补充证明。Mathlib 中常见的 `propext`、`Classical.choice`、`Quot.sound` 属于所采用形式系统的标准基础；若出现 `sorryAx`，则需要检查是否有尚未完成的证明。本例源码没有使用 `sorry`，也没有将目标命题声明为额外公理。依赖核对不能代替对命题、定义及数学原意的人工检查。

## 10 本例的学习收获与核验范围

本例展示了一条完整而可追溯的推理链：

$$\text{素数幂与互素乘积归纳} \Longrightarrow \text{因子和恒等式} \Longrightarrow \text{卷积等式} \Longrightarrow \text{Möbius 反演有限和.}$$

与 3–4–1 例程相比，新增的重点是：归纳原理怎样组织无限多个自然数情形，局部假设怎样用于当前目标，函数等式怎样逐点验证，有限和怎样在卷积语言中统一处理。`rw`、`simp`、`exact`、`calc` 等指令分别承担明确任务，而不是凭一个策略名称替代整段数学论证。

Lean 核验的是实际写出的命题与证明。中文注释、与本书的对应，以及在输入为零处的补值约定是否符合当前用途，仍需读者理解和人工审阅。本例完成的是上述算术恒等式与反演，不涉及 Dirichlet 级数收敛、$\zeta$ 的解析延拓或素数定理。